

torch.addmv#

<https://pytorch.org/docs/stable/generated/torch.addmv.html>

Description:

Performs a matrix-vector product of the matrix `mat` and the vector `vec`. The vector input is added to the final result. If `mat` is a $(n \times m)$ tensor, `vec` is a 1-D tensor of size `m`, then input must be broadcastable with a 1-D tensor of size `n` and out will be 1-D tensor of size `n`. `alpha` and `beta` are scaling factors on matrix-vector product between `mat` and `vec` and the added tensor input respectively. If `beta` is 0, then the content of input will be ignored, and `nan` and `inf` in it will not be propagated. For inputs of type `FloatTensor` or `DoubleTensor`, arguments `beta` and `alpha` must be real numbers, otherwise they should be integers.

Function Signature

```
torch.addmv(input, mat, vec, *, beta=1, alpha=1, out=None) → Tensor#
```

Parameters

Keyword Arguments

Parameters

Keyword

beta (Number, optional) – multiplier for input (β) alpha (Number, optional) – multiplier for mat@vecmat @ vecmat (α) out (Tensor, optional) – the output tensor.

Code Examples

```
>>> M = torch.randn(2)
>>> mat = torch.randn(2, 3)
>>> vec = torch.randn(3)
>>> torch.addmv(M, mat, vec)
tensor([-0.3768, -5.5565])
```

Detailed Documentation

Documentation

Performs a matrix-vector product of the matrix `mat` and the vector `vec`. The vector input is added to the final result.

If `mat` is a $(n \times m)$ tensor, `vec` is a 1-D tensor of size m , then input must be broadcastable with a 1-D tensor of size n and out will be 1-D tensor of size n .

`alpha` and `beta` are scaling factors on matrix-vector product between `mat` and `vec` and the added tensor input respectively.

If `beta` is 0, then the content of input will be ignored, and `nan` and `inf` in it will not be propagated.

For inputs of type `FloatTensor` or `DoubleTensor`, arguments `beta` and `alpha` must be real numbers, otherwise they should be integers.

`input (Tensor)` – vector to be added

`mat (Tensor)` – matrix to be matrix multiplied

`vec (Tensor)` – vector to be matrix multiplied

`beta (Number, optional)` – multiplier for input (β)

`alpha (Number, optional)` – multiplier for `mat@vec` (α)

`out (Tensor, optional)` – the output tensor.

